

BÀI 3. THỰC HÀNH SPRING BOOT – TẠO DỰ ÁN SPRING BOOT CÓ KẾT NỐI CƠ SỞ DỮ LIỆU

Yêu cầu: xây dựng Spring Boot project

Thời lượng: 6 tiết

Mục tiêu:

- Tạo được ứng dụng Java dùng Spring Boot
- Kết nối được cơ sở dữ liệu cho ứng dụng dùng Spring Boot

Nội dung:

Câu 1. Tạo một dự án Spring MVC đầu tiên

a/ Tạo Databaes

```
CREATE DATABASE WebStore;
```

```
GO
```

```
USE WebStore;
```

```
GO
```

b/ Tạo Project Spring Mvc

Truy cập vào link <https://start.spring.io/> để tạo project Spring Mvc

Sau khi truy cập vào link tạo project theo cấu hình như sau

Project	Maven
Language	Java
Spring Boot	3.4.0
Group	WebStore
Artiface	WebStore
Name	WebStore
Package name	webapp
Packing	jar

Java	22
Dependencies	Spring Boot DevTools
	Spring Web
	Thymeleaf
	Spring Data JPA
	MS SQL Server Driver

c/ Configuration database

Vào src/main/resources/application.properties, thêm vào nội dung sau để khai báo kết nối đến CSDL SQL Server

```
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=WebStore;TrustServerCertificate=true;
spring.datasource.username=sa
spring.datasource.password=123
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver
spring.jpa.show-sql: true
spring.jpa.hibernate.naming.implicit-
strategy=org.hibernate.boot.model.naming.ImplicitNamingStrategyLegacyJpaImpl
spring.jpa.hibernate.naming.physical-
strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
```

Lưu ý trong đoạn code trên ta kết nối đến Sql Server với username là sa và password=123, tên CSDL là WebStore

d/ Tạo HomeController

Trong thư mục **main/java/webapp** tạo thêm 2 thư mục là **controllers** và **models**

Sau đó hãy tạo **HomeController** với nội dung như sau:

```
@Controller
public class HomeController {
    @GetMapping("/")
    public String index() {
        return "home/index";
    }
}
```

e/ Tạo view cho action index trong controller HomeController

Vào thư mục **main/resoures/templates** tạo thư mục **home** và file **index.html** trong thư mục home

Trong file **index.html** hãy đặt nội dung bất kỳ ví dụ

```
<h2>Welcome Ladies and Gentlemen</h2>
```

f/ Chạy thử dự án

Nhấp phải vào tên của Project trong cửa sổ Project Explorer, chọn Run As -> Spring Boot App. Sau khi chạy xong, hãy mở trình duyệt web và truy cập **localhost:8080/** để xem kết quả.

Câu 2. Tạo một dự án Spring Boot bằng Eclipse có khai báo kết nối cơ sở dữ liệu MySQL

a/ Cài đặt Spring Tool cho Eclipse: Help -> Eclipse Marketplace, gõ vào ô Find từ khóa tìm kiếm “Spring” và nhấn phím Enter.

Chọn cài đặt Spring Tool 4 và thực hiện đủ các bước cài đặt.

b/ Tạo Spring Boot Project mới:

- Mở Eclipse, chọn menu File ->New -> Other...
- Mở rộng mục Spring Boot, chọn Spring Starter Project, nhập Next.
Thiết lập metadata như sau:

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

- Nhấp nút Next
- Lựa chọn các thành phần phụ thuộc như trong hình sau:

New Spring Starter Project Dependencies

Spring Boot Version:

Frequently Used:

☒ MySQL Driver ☒ Spring Boot DevTools ☒ Spring Data JPA

☒ Spring Web ☒ Thymeleaf

Available:

Type to search dependencies

Selected:

X Spring Boot DevTools

X Spring Data JPA

X MySQL Driver

X Thymeleaf

X Spring Web

- Nhấp Next, Finish.

c/ Tạo cơ sở dữ liệu WebStore trên MySQL

```
CREATE DATABASE WebStore;
```

```
GO
```

```
USE WebStore;
```

```
GO
```

d/ Khai báo cấu hình kết nối cơ sở dữ liệu cho Spring boot project tại file application.properties

```
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:mysql://localhost:3306/WebStore
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.show-sql: true
spring.jpa.hibernate.naming.implicit-
strategy=org.hibernate.boot.model.naming.ImplicitNamingStrategyLegacyJpaImpl
spring.jpa.hibernate.naming.physical-
strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
```

e/ Tạo HomeController

Trong thư mục **main/java/webapp** tạo thêm 2 thư mục là **controllers** và **models**

Sau đó ta tạo HomeController với nội dung như sau:

```
@Controller
public class HomeController {
    @GetMapping("/webstore")
    public String index() {
        return "home/index";
    }
}
```

f/ Tạo view cho action index trong controller HomeController

Vào thư mục **main/resources/templates** tạo thư mục **home** và file **index.html** trong thư mục **home** với nội dung tự biên tập.

g/ Chạy thử dự án

Nhấp phải vào tên của Project trong cửa sổ Project Explorer, chọn Run As -> Spring Boot App. Sau khi chạy xong, hãy mở trình duyệt web và truy cập **localhost:8080/webstore** để xem kết quả.

Câu 3: Thực hành các hoạt động quản lý dữ liệu với CRUD (Create, Read, Update, Delete)

a/ Tạo bảng Category

Sử dụng lệnh sau để tạo bảng Category như sau

```
--DROP TABLE Category;
GO
CREATE TABLE Category(
    CategoryId SMALLINT NOT NULL PRIMARY KEY IDENTITY(1, 1),
    CategoryName NVARCHAR(64) NOT NULL
);
GO
SET IDENTITY_INSERT Category ON;
INSERT INTO Category (CategoryId, CategoryName) VALUES
    (1, 'Men shoes'),
    (2, 'Women shoes'),
    (3, 'Hand bag'),
    (4, 'fashion accessories');
SET IDENTITY_INSERT Category ON;
```

b/ Tạo Model

Tạo class Category

Vào thư mục **models** trong **java/webapp** tạo **Category.java** và viết code như sau

```
@Entity(name = "Category")
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "CategoryId")
    private short id;
    @Column(name = "CategoryName")
    private String name;
    public Category(){
    }
    public Category(short id, String name){
        this.id = id;
        this.name = name;
    }
    public short getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public void setId(short id) {
        this.id = id;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

c/ Tạo interface CategoryRepository

Vào models tạo tiếp CategoryRepository.java viết code như sau

```
public interface CategoryRepository extends CrudRepository<Category,Short> {
}
```

d/ Tạo CategoryController

Trong thư mục controllers ta tạo file CategoryController.java như sau

```
@Controller
public class CategoryController {

    @Autowired
    CategoryRepository repository;

    @GetMapping("/category")
    public String index(Model model){
        model.addAttribute("list", repository.findAll());
        return "category/index";
    }

    @GetMapping("/category/delete/{id}")
    public String delete(@PathVariable("id") short id){
        repository.deleteById(id);
        return "redirect:/category";
    }

    @GetMapping("/category/add")
    public String add(){
        return "category/add";
    }

    @PostMapping("/category/add")
```



```

public String add(Category obj){
    repository.save(obj);
    return "redirect:/category";
}

@GetMapping("/category/edit/{id}")
public String edit(@PathVariable("id") short id, Model model){
    model.addAttribute("o", repository.findById(id).get());
    return "category/edit";
}

@PostMapping("/category/edit/{id}")
public String edit(@PathVariable("id") short id, Category obj){
    repository.save(obj);
    return "redirect:/category";
}
}

```

e/ Tạo View cho CategoryController

View index.html

Trong thư mục **templates**, tạo thư mục category, rồi tạo index.html với nội dung sau:

```

<a href="/category/add">Add</a>
<table>
  <thead>
    <tr>
      <th>Id</th>
      <th>Name</th>
      <th>Command</th>
    </tr>
  </thead>

```

```

<tbody>
  <tr th:each="item : ${list}">
    <td th:text="${item.id}"></td>
    <td th:text="${item.name}"></td>
    <td>
      <a th:href="@{/category/edit/{id}(id=${item.id})}">Edit</a>
      <a th:href="@{/category/delete/{id}(id=${item.id})}">Delete</a>
    </td>
  </tr>
</tbody>
</table>

```

f/ View add.html

Trong templates/category tạo tiếp add.html, rồi nhập nội dung sau:

```

<form method="post">
  <div>
    <label for="name">Name</label>
    <div>
      <input type="text" name="name" id="name">
    </div>
  </div>
  <div>
    <button>Save Changes</button>
  </div>
</form>

```

g/ View edit.html

```

<form method="post">

```

```

<div>
    <label for="name">Name</label>
    <div>
        <input type="text" name="name" id="name" th:value="${o.name}">

    </div>
</div>
<div>
<div>
    <button>Save Changes</button>
</div>
</form>

```

Câu 4: Upload File

a/ Cấu hình upload file

Để upload vào thư mục đang chạy web ta cần tạo cấu hình chuyển đổi upload file trên thư mục web đang chạy trên server bằng cách tạo **MvcConfig.java** trong **webapp** như sau:

```

@Configuration
public class MvcConfig implements WebMvcConfigurer{

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {

        String dirName = "images";

        Path dir = Paths.get("images");

        String uploadPath = dir.toFile().getAbsolutePath();

        registry.addResourceHandler("/" + dirName + "/*").addResourceLocations("file://"
+ uploadPath + "/");

    }
}

```

b/ Tạo Service Upload File

Tạo thêm thư mục **services** trong **src/main/java/webapp**

Tạo thư mục **images** trong **src/main/resources/static**

Tạo class **StorageProperties**

Tạo **StorageProperties.java** trong thư mục services để khai báo đường dẫn images trên thư mục web resource như sau:

```
@ConfigurationProperties("storage")
public class StorageProperties {
    private String location = "./src/main/resources/static/images";
    public String getLocation() {
        return location;
    }
    public void setLocation(String location) {
        this.location = location;
    }
}
```

Sau khi tạo class **StorageProperties** xong ta cấu hình chạy khi webapp được khởi tạo trong class **WebStoreApplication** như sau:

```
@EnableConfigurationProperties(StorageProperties.class)
public class WebStoreApplication {

    public static void main(String[] args) {
        SpringApplication.run(WebStoreApplication.class, args);
    }
}
```

Trong code trên ta chỉ thêm phần code **@EnableConfigurationProperties(StorageProperties.class)**, để biết được đường dẫn lưu trữ file trong resources

c/ Tạo class Helper

Tạo Helper.java trong thư mục services để xử lý upload file với nội dung sau

```
public class Helper {

    public static String saveFile(String uploadDir, MultipartFile file, int len) throws
    IOException {

        Path path = Paths.get(uploadDir);

        if(!Files.exists(path)) {

            Files.createDirectories(path);

        }

        String name = file.getOriginalFilename();

        String ext = name.substring(name.lastIndexOf('.'));

        String fileName = randomString(len - ext.length()) + ext;

        Path filePath = path.resolve(fileName);

        file.transferTo(filePath);

        return fileName;

    }

    public static String randomString(int len){

        StringBuilder sb = new StringBuilder(len);

        String pattern = "1234567890qwertyuiopasdfghjklzxcvbnm";

        Random random = new Random();

        for(int i = 0; i < len; i++){

            sb.append(pattern.charAt(random.nextInt(pattern.length())));

        }

        return sb.toString();

    }

}
```

Trong class trên có phương thức **randomString** để tạo chuỗi ngẫu nhiên với chiều dài truyền vào là **len**

Đối với phương thức **saveFile** xử lý với mục đích lưu trữ vào thư mục, trong quá trình lưu file cần tạo chuỗi ngẫu nhiên nhằm tránh trường hợp trùng tên tập tin, sẽ bị ghi đè

d/ Cấu hình kích thước file upload

Trường hợp kích thước file vượt quá mặc định cho phép trong Spring MVC ta cần cấu hình kích thước tập tin tối đa có thể upload trong application.properties như sau

```
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
```

e/ Tạo model

Tạo class Image

Tạo Image.java trong thư mục models để lưu trữ các thông tin như đường dẫn, kiểu tập tin, kích thước như trong code sau:

```
@Entity
public class Image {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "ImageId")
    private int id;

    @Column(name = "ImageUrl")
    private String url;

    private long size;

    private String type;

    public Image() {
    }

    public Image(int id, String url, long size, String type) {
        super();
        this.id = id;
```

```
this.url = url;

this.size = size;

this.type = type;
}

public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public String getUrl() {
    return url;
}

public void setUrl(String url) {
    this.url = url;
}

public long getSize() {
    return size;
}

public void setSize(long size) {
    this.size = size;
}

public String getType() {
    return type;
}

public void setType(String type) {
    this.type = type;
}
```

```
}
}
```

Class ImageRepository

Tạo tiếp ImageRepository.java trong models với nội dung sau:

```
public interface ImageRepository extends CrudRepository<Image, Integer>{
}
```

f/ Tạo Controller

Tạo UploadController

Trong thư mục controllers tạo UploadController.java với nội dung sau:

```
@Controller
public class UploadController {

    @Autowired
    private ImageRepository repository;

    @GetMapping("/upload")
    public String index(Model model){
        model.addAttribute("list", repository.findAll());
        return "upload/index";
    }

    @GetMapping("/upload/add")
    public String add(){
        return "upload/add";
    }

    @PostMapping("/upload/add")
    public String add(MultipartFile f){
        try {
```



```

        String fileName = Helper.saveFile("images", f, 32);
        Image image = new Image(0, fileName, f.getSize(), f.getContentType());
        repository.save(image);
        return "redirect:/upload";
    } catch (IOException e) {
        e.printStackTrace();
    }
    return "upload/add";
}
}

```

g/ Tạo Views

Tạo thư mục **upload** trong **templates**

Tạo view index

Trong thư mục upload tạo index.html với nội dung sau để hiển thị thông tin upload file như sau:

```

<a href="/upload/add">Add</a>
<table>
  <tbody>
    <tr th:each="o : ${list}">
      <td th:text="${o.id}"></td>
      <td>
        
      </td>
      <td th:text="${o.type}"></td>
    </tr>
  </tbody>
</table>

```

Tạo view add

Trong thư mục upload tiếp tục tạo add.html để upload file

```
<form enctype="multipart/form-data" method="post">
  <div>
    <label for="f">Image</label>
    <div>
      <input type="file" name="f" id="f" accept="image/*">
    </div>
  </div>
  <div>
    <button>Upload</button>
  </div>
</form>
```